



VET-C 邊緣模型建置指南(續)

(RFC 2000 ~6999)

從感測器輸入到無人機馬達控制的 AGI 閉環完整實作流程，涵蓋 10 個核心步驟，適用於 Jetson Orin Nano 等邊緣設備部署。

VET-C MODEL

PYTORCH 實作

邊緣 AI

RFC-0000 VET-C Master RFC Tree

Version: 0.0 | Status: Index Document
本文件為整個 VET-C RFC 系列的根索引，定義所有 RFC 的編號體系與演化路徑。所有後續 RFC 均以本文件為基礎延伸。

VET-C 的 RFC 系列採用類似 Linux Kernel、ROS2、LLVM 的工程憲法架構，以千位數字區分不同層次的規格文件，形成清晰的演化路徑與可擴充的工程體系。

RFC 範圍	類別	說明	目前狀態
RFC-0000 ~ 0016	Theory RFC	系統架構、Tensor 契約、損失函數、Regime、Terrain、Memory、Dream、密度流、扭量幾何、具身認知、UCFT 統一理論	✅ 完成
RFC-1000 ~ 1999	Implementation RFC	Codex Sprint 工程指令：Repository、Core Types、Encoder、Fusion、RSSM、Prediction、Regime Engine、Terrain Generator、Policy、Runtime Loop	✅ RFC-1000~1015 完成
RFC-2000 ~ 2999	Training RFC	訓練管線規格：損失函數實作、訓練策略、Checkpoint、TensorBoard、多階段訓練排程	👉 待開發
RFC-3000 ~ 3999	Dataset RFC	資料集規格：AirSim 資料收集、資料格式、前處理管線、資料增強、Dataset Loader	👉 待開發
RFC-4000 ~ 4999	Simulation RFC	模擬環境規格：AirSim 環境設定、PX4 整合、感測器模擬、場景生成、評估指標	👉 待開發
RFC-5000 ~ 5999	Memory RFC	記憶系統規格：Event Discovery、Bubble/Pattern/Impression 壓縮、Semantic Graph、Memory Replay、Dreaming	👉 待開發
RFC-6000 ~ 6999	Dream RFC	夢境系統規格：Counterfactual Simulation、Dream Tree、H+ Grasp、Insight、Dream Scheduler	👉 待開發
RFC-7000 ~ 7999	Density Flow RFC	密度流規格：Fokker-Planck 實作、Basin Boundary、CPA、Future Density Cloud、Trajectory Pruning	👉 待開發
RFC-8000 ~ 8999	Meta-Regime RFC	後設認知規格：五行十向 Codebook、Regime Transition Matrix、Regime Graph、Consciousness Trajectory	👉 待開發
RFC-9000 ~ 9999	UCFT Final RFC	統一認知場論： $dE/dt = -\nabla \Psi + \tau + F_memory + F_dream + F_affordance$ 完整實作、Fiber Bundle、Twistor	👉 待開發

👉 VET-C Master RFC Tree 定義了整個系統從理論到工程的完整演化路徑。目前 RFC-0000~0016（Theory）與 RFC-1000~1015（Implementation）已完成，構成 VET-C v0.1 MVP 的完整基礎。

RFC-2001

Trainer Core Specification: 開始建立整個訓練系統。

RFC-2000 Series

這一系列的目的只有一個：

讓 RFC-1000~1015 所建立的 VET-C 架構真正學起來。

而且這一系列最重要的一個思想是：

VET-C 不做 End-to-End Training

而是：

```
graph TD; A[Perception] --> B[World Model]; B --> C[Regime]; C --> D[Terrain]; D --> E[Policy]; E --> F[Joint Fine-Tuning];
```

Perception
↓
World Model
↓
Regime
↓
Terrain
↓
Policy
↓
Joint Fine-Tuning

因此 RFC-2000 的順序非常重要。

RFC-2001 Trainer Core Specification

Version: 1.0 | Status: Implementation RFC | Target: vetc/trainer/

1 Goal

建立整個 VET-C 的訓練骨架。所有訓練都透過 Trainer 統一管理。
未來以下模組都共用同一個 Trainer：

- World Model Training
- Regime Training
- Terrain Training
- Policy Training

2 Target Files

```
vetc/trainer/  
├── trainer.py  
├── checkpoint_manager.py  
├── logger.py  
├── experiment.py  
├── state_dict_utils.py  
├── tests/  
│   └── test_trainer.py
```

3 Architecture

```
Trainer  
├── model  
├── optimizer  
├── scheduler  
├── checkpoint_manager  
├── logger  
└── experiment
```

4 Trainer State

```
@dataclass  
class TrainerState:  
    epoch: int  
    global_step: int  
    best_loss: float  
    current_loss: float  
    learning_rate: float  
    device: str
```

5 ExperimentConfig

```
@dataclass  
class ExperimentConfig:  
    experiment_name: str  
    save_dir: str  
    batch_size: int  
    epochs: int  
    learning_rate: float  
    device: str
```

6 Logger

建立 class MetricLogger，支援：

- log(metric name, value, step)
- log dict({"loss": 0.12, "kl": 0.003})
- save_csv() → 輸出 metrics.csv

7 Checkpoint Manager

建立 class CheckpointManager，支援：

- save(epoch, model, optimizer, trainer_state) → 生成 checkpoint epoch_001.pt
- load() → 恢復 model.load_state_dict() / optimizer.load_state_dict()
- latest_checkpoint() → 取得最新檔案

8 Experiment Directory

自動建立：

```
experiments/  
├── exp_001/  
│   ├── checkpoints/  
│   ├── logs/  
│   └── configs/
```

RFC-2001 是訓練系統的作業系統（Training OS）。

接下來真正開始建立：

VET-C Loss Graph

這一份非常重要。

因為 VET-C 並不是單一 loss。

而是一個 Layered Loss Architecture。

未來：

- World Model
- Regime
- Terrain
- Policy
- Memory
- Dream

全部都會掛在這個 Loss Registry 上。

RFC-2002 Loss Registry System Specification

Version: 1.0 | Status: Implementation RFC | Target: vetc/losses/

1 Philosophy

禁止以下散落在 code 中的寫法：

```
loss = loss1 + loss2 + loss3
```

必須集中管理：

```
LossRegistry
↓
WorldLoss
↓
RegimeLoss
↓
TerrainLoss
↓
PolicyLoss
↓
TotalLoss
```

2 Target Files

```
vetc/losses/
├── base_loss.py
├── loss_registry.py
├── loss_outputs.py
├── world_losses.py
├── regime_losses.py
├── terrain_losses.py
├── policy_losses.py
├── total_loss.py
├── tests/
│   └── test_loss_registry.py
```

3 BaseLoss

建立 class BaseLoss(nn.Module) , API :

- forward(...) → 回傳 LossOutput

4 LossOutput

```
@dataclass
class LossOutput:
    total_loss: Tensor
    metrics: dict[str, float]
    tensors: dict[str, Tensor]
```

例如：

```
metrics = {
    "reconstruction": 0.12,
    "kl": 0.003
}
```

5 LossRegistry

建立 class LossRegistry , 功能：

- register()
- get()
- compute()

6 World Losses

- Reconstruction Loss : $MSE(x_{\hat{}} , x)$
- KL Loss : 對 posterior / prior 計算 $KL(p||q)$
- RSSM Consistency Loss : $MSE(z_{post} , z_{prior})$
- Total World Loss :

$$L_{world} = L_{recon} + \beta \cdot KL + \gamma \cdot consistency$$

7 Regime Loss

包含：

- VQ Loss (L_{commit})
- Codebook Loss ($L_{codebook}$)
- Dead Code Penalty
- Entropy Loss (避免 collapse)

總：

RFC-2001、2002 完成後，其實已經建立了：

Trainer OS

+

Loss Graph

接下來是整個 Training System 的心臟：

RFC-2003 Optimizer & Scheduler System Specification

Version: 1.0 | Status: Implementation RFC | Target: vetc/trainer/

1 Philosophy

禁止以下散落在各個 training code 的寫法：

```
optimizer = Adam(...)
scheduler = ...
```

必須集中管理：

```
OptimizerFactory
↓
SchedulerFactory
↓
EMA
↓
GradientClip
↓
OptimizerManager
```

2 Target Files

```
vetc/trainer/
├── optimizer_factory.py
├── scheduler_factory.py
├── ema.py
├── gradient_utils.py
├── optimizer_manager.py
├── tests/
│   └── test_optimizer_system.py
```

3 OptimizerFactory

建立 class OptimizerFactory，支援：

- AdamW
- SGD
- RMSProp

API：build(model, config)

4 Parameter Groups

支援不同模組使用不同 learning rate：

```
encoder_lr: 1e-4
rssm_lr: 3e-4
regime_lr: 1e-4
terrain_lr: 1e-5
policy_lr: 3e-5
```

5 SchedulerFactory

支援：

- Cosine Annealing
- Warmup Cosine
- StepLR
- Constant

API：build(optimizer, config)

6 Warmup

定義 warmup_steps，學習率公式：

```
lr = lr_max * step / warmup_steps
```

之後進入 Cosine decay。

7 EMA

建立 class EMA，保存 shadow_model，更新公式：

```
 $\theta_{ema} = \beta \cdot \theta_{ema} + (1 - \beta) \cdot \theta$ 
```

預設： $\beta = 0.999$

8 Gradient Utils

建立以下工具函數：

- cliio eradients()

RFC-2004 Mixed Precision & Runtime Acceleration Training Specification

Version: 1.0 | Status: Implementation RFC | Target: vetc/trainer/amp.py, compile_utils.py, profiler.py

1 Goal

建立 VET-C 訓練加速層：

```
Trainer
↓
AMP
↓
GradScaler
↓
torch.compile
↓
Profiler
```

目的：讓 VET-C 在 GPU / Jetson 上訓練與推論更穩定、更快。

2 Target Files

```
vetc/trainer/
├── amp.py
├── compile_utils.py
├── profiler.py
tests/
├── test_amp_compile_profiler.py
```

3 AMP Manager

```
class AMPManager:
    def __init__(self, enabled=True, dtype="float16"):
        ...

    def autocast(self):
        ...

    def backward(self, loss, optimizer_manager):
        ...
```

4 GradScaler

- 若 CUDA 可用：使用 torch.cuda.amp.GradScaler
- 若 CPU / MPS：使用 disabled scaler

5 AMP Rule

訓練 step 寫法：

```
with amp.autocast():
    loss_output = loss_fn(...)
    loss = loss_output.total_loss

amp.backward(loss, optimizer_manager)
```

6 torch.compile

```
def maybe_compile(model, enabled=True, mode="default"):
    ...
```

- 若 PyTorch 支援：回傳 torch.compile(model)
- 若不支援：回傳原 model

7 Profiler

```
class TrainingProfiler:
    def start(self): ...
    def stop(self): ...
    def step(self): ...
    def summary(self): ...
```

記錄指標：

- step time ms
- forward time ms
- backward_time_ms
- ootimizer time ms
- memory_allocated_mb

8 Unit Tests

RFC-2005 World Model Training Pipeline Specification

Version: 1.0 | Status: Implementation RFC | Target: vetc/trainer/train_world_model.py

1 Goal

讓 VET-C 第一次真正「學世界」。

訓練管線：

```
ObservationPacket Sequence
↓
ObservationLayer
↓
DualPerceptionEncoder
↓
RegimeConditionedFusion
↓
UnifiedRSSM
↓
PredictionErrorEvaluator
↓
World Loss
↓
Optimizer
```

2 Target Files

```
vetc/trainer/
├── train_world_model.py
tests/
├── test_train_world_model.py
```

3 Input Batch

```
batch = {
    "obs_seq": list[ObservationPacket], # length T
    "action_seq": torch.Tensor,        # [B, T, 4]
    "done_seq": torch.Tensor,          # [B, T]
}
```

MVP 中 action_seq 先保留但 RSSM 暫不使用。

4 WorldModelTrainer

```
class WorldModelTrainer(Trainer):
    def train_step(self, batch) -> LossOutput:
        ...
```

5 Sequence Forward

```
latent_state = None
losses = []

for t in range(T):
    obs_tensor = observation_layer(obs_seq[t])
    enc = encoder(obs_tensor)
    fusion = fusion(enc, regime=None)
    latent_state = rssm(fusion, latent_state)
    pred = prediction(latent_state)
    loss_t = world_loss(latent_state, pred)
    losses.append(loss_t.total_loss)

loss = mean(losses)
```

6 Teacher Forcing

MVP：每一步都使用真實 obs_t 編碼產生 posterior。未來版本才加入 latent rollout。

7 Required Losses

使用 RFC-2002：

- `KLLoss`
- `RSSMConsistencyLoss`
- `PredictionSurpriseLoss`

MVP 總損失：

```
L_world = kl_loss + consistency_loss + surprise_loss
```

8 BPTT

RFC-2006 Regime Training Pipeline Specification

Version: 1.0 | Status: Implementation RFC | Target: vetc/trainer/train_regime.py

1 Goal

訓練 VET-C 的 Regime Engine，使 latent state 被穩定離散化為：

- R_ext
- R_int
- R_mission
- R_meta

MVP 先訓練 R_ext、R_int、R_mission，R_meta 先保持 zero placeholder。

2 Target Files

```
vetc/trainer/  
├── train_regime.py  
tests/  
├── test_train_regime.py
```

3 Input Batch

```
batch = {  
    "latent_state": LatentState,  
    "prediction": PredictionOutput,  
}
```

也可以從 frozen world model 產生：

ObservationPacket Sequence → Encoder → RSSM → Prediction → Regime Training

MVP 建議先用已產生的 latent batch 測試。

4 RegimeTrainer

```
class RegimeTrainer(Trainer):  
    def train_step(self, batch) -> LossOutput:  
        ...
```

5 Forward Flow

```
LatentState + PredictionOutput  
↓  
RegimeEngineVQVAE  
↓  
RegimeTensor  
↓  
Regime Loss
```

6 Loss

使用 RFC-2002：

- VQ commitment loss
- codebook loss
- usage entropy loss
- dead code penalty

MVP 總損失：

$$L_{\text{regime}} = L_{\text{commit}} + L_{\text{codebook}} - \lambda_{\text{usage}} * H_{\text{usage}}$$

7 Metrics

- regime_loss
- commit_loss
- codebook_loss
- usage_entropy
- dead_code_count

8 Frozen Modules

Regime training 階段凍結：

- Encoder frozen
- RSSM frozen
- Prediction frozen

只訓練：

- RegimeEngineVQVAE
- VectorQuantizer

RFC-2008 Policy Training Pipeline Specification

Version: 1.0 | Status: Implementation RFC | Target: vetc/trainer/train_policy.py

1 Goal

訓練 VET-C 的第一版 Policy :

```
TerrainOutput
↓
FastGradientPolicy
↓
ActionCommand
```

MVP 不做 RL , 只做 imitation / supervised action learning °

2 Target Files

```
vetc/trainer/
└── train_policy.py
tests/
└── test_train_policy.py
```

3 Input Batch

```
batch = {
    "terrain": TerrainOutput,
    "action_target": torch.Tensor, # [B, 4]
}
```

4 PolicyTrainer

```
class PolicyTrainer(Trainer):
    def train_step(self, batch) -> LossOutput:
        ...
```

5 Forward Flow

```
TerrainOutput
↓
FastGradientPolicy
↓
ActionCommand
↓
Policy Loss
```

6 Loss

```
L_action = MSE(action_pred, action_target)
```

Action target 必須符合 :

- shape: [B, 4]
- range: [-1, +1]

7 Metrics

- policy loss
- action mse
- action_abs_mean
- action_saturation_rate

8 Frozen Modules

Policy training 階段凍結 :

- Encoder frozen
- RSSM frozen
- Regime frozen
- Terrain frozen

只訓練 :

- FastGradientPolicy
- ActionDecoder

9 Unit Tests

測試 :

- dummy TerrainOutput + action_target 可 train one step

RFC-3001

Dataset System

到這裡其實整個 VET-C 已經從

1000 Architecture

2000 Training

進入真正開始產生經驗的地方：

RFC-3000 Dataset System

這一系列的重要性甚至不亞於模型本身。

因為：

World Model 的能力上限 = Dataset 的品質。

我建議整個 Dataset Tree：

RFC-3000 Dataset Master

3001 Dataset Format

3002 Episode Storage

3003 Sequence Dataset

3004 Augmentation

3005 Normalization

3006 Train-Val-Test Split

3007 Sequence Sampler

3008 Replay Buffer

3009 Online Dataset

3010 Dataset Registry

RFC-3000 Dataset System Master Specification

Version: 1.0 | Status: Master RFC | Target: vetc/dataset/

Goal

建立完整的 Dataset Pipeline :

```
Raw Sensor
↓
Episode
↓
Sequence
↓
Dataset
↓
Replay Buffer
↓
Trainer
```

支援 :

- Offline Dataset
- Online Dataset
- AirSim Dataset
- PX4 Dataset
- Real Drone Dataset
- Memory Dataset

RFC-3001 Episode Dataset Format Specification

Version: 1.0 | Target: vetc/dataset/

1 Goal

建立 VET-C 統一 Episode 格式。
禁止各種不同格式，必須統一為：

```
episode/
├── 000001/
│   ├── rgb/
│   ├── imu.npy
│   ├── battery.npy
│   ├── motor.npy
│   ├── action.npy
│   ├── reward.npy
│   ├── done.npy
│   └── meta.yaml
```

2 Folder Structure

```
dataset/
├── episode_000001/
│   ├── rgb/
│   │   ├── 000000.png
│   │   └── 000001.png
│   │   └── ...
│   ├── depth/
│   │   ├── flow/
│   │   ├── imu.npy
│   │   ├── battery.npy
│   │   ├── motor_state.npy
│   │   ├── action.npy
│   │   ├── reward.npy
│   │   ├── done.npy
│   │   ├── timestamp.npy
│   │   └── meta.yaml
```

3 Tensor Shapes

欄位	Shape
IMU	[T, 6]
Battery	[T, 1]
Motor	[T, 4]
Action	[T, 4]
Done	[T]

4 Metadata

```
episode_id:
env:
date:
fps:
length:
collision:
success:
```


RFC-4001 Environment + Body + Physics

Simulation System Master Specification

很好。

其實到了這裡，整個 VET-C 專案開始從「神經系統」走向「身體」。

前面的：

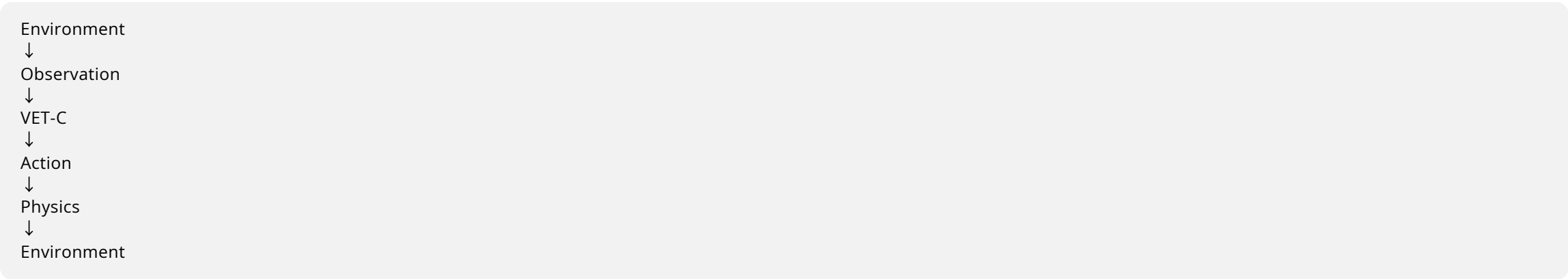
- 1000 Architecture
- 2000 Training
- 3000 Dataset

都是大腦。

從 RFC-4000 開始，真正開始建立：

Environment + Body + Physics

形成：



形成真正的 Embodied AI。

RFC-4000 Simulation System Master Specification

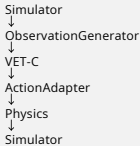
Version: 1.0 | Status: Master RFC | Target: vetc/simulator/

Goal

建立統一模擬介面：

- AirSim
- PX4
- Real Drone

形成完整閉環：



RFC Tree

RFC-4000 Simulation Master

- 4001 Environment Interface
- 4002 AirSim Connector
- 4003 Observation Generator
- 4004 Action Adapter
- 4005 Runtime Bridge
- 4006 Obstacle Environment
- 4007 Corridor Environment
- 4008 Disturbance Environment
- 4009 Reward Logger
- 4010 Benchmark Suite

RFC-4001 Environment Interface Specification

Version: 1.0 | Target: vetc/simulator/

Goal

統一以下環境：AirSim、PX4、Gazebo、IsaacSim、Real Drone

建立 class BaseEnvironment，API：

- reset() → 回傳 ObservationPacket
- step(ActionCommand) → 回傳 ObservationPacket, reward, done, info
- close()
- render()
- seed()

Unit Test：DummyEnvironment

RFC-4002 AirSim Connector Specification

Target: vetc/simulator/airsim_connector.py

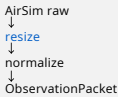
建立 class AirSimConnector(BaseEnvironment)，內部使用 airsims.MultirotorClient()。

支援：arm() / takeoff() / reset() / hover() / land()

Observation 取得：RGB、Depth、Flow、IMU、Position、Velocity、Battery → 形成 ObservationPacket

RFC-4003 Observation Generator Specification

建立 class ObservationGenerator，流程：



Tensor Shapes：

欄位	Shape
RGB	[B, 3, 256, 256]
Depth	[B, 1, 256, 256]
Flow	[B, 2, 256, 256]
IMU	[B, 32, 6]
Battery	[B, 22, 1]

RFC-5001

TraEpisodic Memory System

其實到了這裡，整個 VET-C 才剛剛從 DreamerV3 等 World Model 開始分岔。

因為：

1000 Architecture
2000 Training
3000 Dataset
4000 Simulation

只能得到：

VET-C v0.1

它是一個會飛的 Embodied Agent。

但還沒有：

- 記憶
- 印象
- 經驗
- 聯想
- 夢境
- H+

而這些全部在：

RFC-5000 Episodic Memory System

開始出現。

RFC-5000 Memory System Master Specification

Version: 1.0 | Status: Master RFC | Target: vetc/memory/

Goal

建立 VET-C Hippocampus，形成：

Raw Experience
↓
Event
↓
Bubble
↓
Pattern
↓
Impression
↓
Memory Block
↓
Retrieval

RFC Tree

RFC-5000 Memory Master

- 5001 Event Detector
- 5002 Event Encoder
- 5003 Bubble Memory
- 5004 Pattern Memory
- 5005 Impression Memory
- 5006 Memory Block
- 5007 Importance Scoring
- 5008 Retrieval Engine
- 5009 Semantic Memory Graph
- 5010 Long-term Compression

RFC-5001 Event Detector Specification

Goal

將連續 stream 分割成 Event。

輸入：Observation / LatentState / Prediction / Regime / Terrain

輸出：Event + EventScore

定義：

- Surprise： $S = -\log p(z)$
- Novelty
- Flow density
- Value / Terrain Value
- Regime Transition (Codebook switching)

公式：

EventScore = α ·Surprise + β ·Novelty + γ ·Value + δ ·Transition

超過 threshold → 建立 Event。

Output：event_token [B]

RFC-5002 Event Encoder Specification

建立 class EventEncoder。

輸入：LatentState / Terrain / Regime

產生：

- event_embedding [B, 128]
- $VQ \rightarrow event_token$ [B]

Codebook：K = 512

RFC-5003 Bubble Memory Specification

短期記憶，形成：

3~10 events
↓
Bubble

類似 Working Memory。

Structure：

- bubble_embedding [256]
- bubble_token

Codebook：1024

RFC-5004 Pattern Memory Specification

RFC 6000~9999 從工程層進入認知層 — 總論

模型從會記憶變成會想像

從這裡開始，VET-C 才真正開始離開 DreamerV3、JEPA、MemGPT 等現有架構。

建議先不要進入 RFC-6001，而是先建立：

6000 Dream System Tree

7000 Density Flow Tree

8000 Meta-Regime Tree

9000 UCFT Tree

因為這四棵樹共同構成：

Cognitive Layer

而 1000~5000 則只是：

Neural Layer

RFC-6000 Dream System Master Specification

Version: 1.0 | Status: Master RFC | Target: vetc/dream/

Goal

建立 **Counterfactual Imagination System**，形成：

```
Memory
↓
Dream Scheduler
↓
Trajectory Rollout
↓
Dream Tree
↓
Counterfactual
↓
Insight
↓
H+
↓
Memory
```

RFC Tree

6001 Dream Scheduler
6002 Dream Generator
6003 Trajectory Rollout
6004 Dream Tree
6005 Branch Pruning
6006 Counterfactual Engine
6007 Dream Replay
6008 Insight Discovery
6009 H+ Candidate Generator
6010 Dream-Memory Integration

Dream Philosophy

DreamerV3：

latent rollout

VET-C：

memory-guided counterfactual imagination

不是 prediction，而是：

alternative futures。

RFC-7000 Density Flow System Master Specification

Version: 1.0 | Status: Master RFC | Target: vetc/density_flow/

Goal

建立 **Basin Boundary Intelligence**，形成：

```
Future Trajectories
↓
Density Flow
↓
Fokker-Planck
↓
CPA
↓
Boundary
↓
Basin
↓
Danger Set
↓
Action
```

這是 Enactive VET-C 最重要的一層。

RFC Tree

7001 Density Estimator
7002 Flow Model
7003 Fokker-Planck Solver
7004 Future Trajectory Ensemble
7005 Convex Polytope Analyzer
7006 Basin Boundary Detector
7007 Danger Set Encoder
7008 CPA Metric
7009 Boundary Pressure
7010 Flow Memory Integration

RFC-6001 Dream Scheduler Specification

Version: 1.0 | Status: Implementation RFC | Target: vetc/dream/dream_scheduler.py

1 Goal

建立 VET-C 的 Dream 觸發機制。
Dream 不是隨機發生，而是由以下條件觸發：

```
Idle Period
↓
Dream Scheduler
↓
Dream Session
```

2 Target Files

```
vetc/dream/
├── dream_scheduler.py
├── tests/
│   └── test_dream_scheduler.py
```

3 Trigger Conditions

建立 class DreamScheduler，觸發條件：

- **Idle**：無新 Observation 輸入超過 N 步
- **Surprise Spike**：EventScore 超過 threshold
- **Regime Transition**：Codebook switching 發生
- **Manual**：外部呼叫 trigger()

4 DreamSession

```
@dataclass
class DreamSession:
    trigger_reason: str
    start_time: float
    memory_seeds: list[MemoryBlock]
    max_steps: int
    temperature: float
```

5 Scheduler State

```
@dataclass
class SchedulerState:
    idle_steps: int
    last_surprise: float
    last_regime: int
    dream_count: int
    is_dreaming: bool
```

6 Core API

建立 class DreamScheduler：

- step(obs, event_score, regime) → 更新 SchedulerState
- should_dream() → bool
- create_session() → DreamSession
- end_session()

7 Memory Seed Selection

從 MemoryBlock 中選取 seed：

- 優先選 high importance_score
- 優先選 recent + rare
- 數量：top-k（預設 k=5）

8 Config

```
dream_scheduler:
    idle_threshold: 50
    surprise_threshold: 2.0
    max_dream_steps: 200
    temperature: 1.0
    seed_k: 5
```

9 Unit Tests

測試：

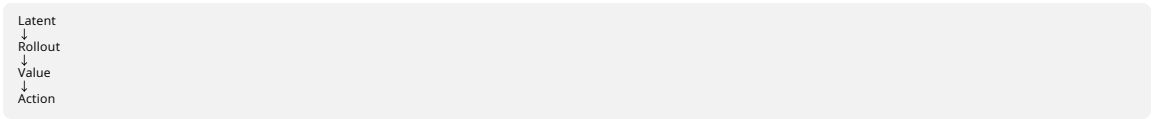
- idle trigger 正常
- surprise trigger 正常
- DreamSession 建立正常
- SchedulerState 更新正常
- should_dream() 回傳正確

10 Codex Prompt

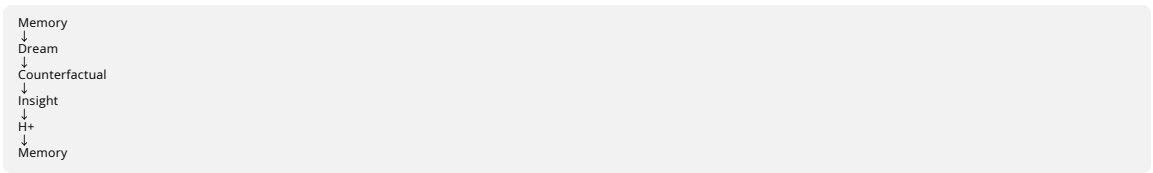
RFC-6000 Dream System

這一層開始，VET-C 已經不是 DreamerV3 。

DreamerV3：



而 VET-C：



形成真正的認知循環。

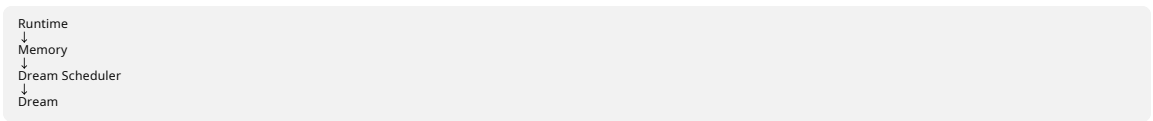
RFC-6001 Dream Scheduler Specification

Version: 1.0 | Target: vetc/dream/

Goal

決定：什麼時候開始做夢？

Dream 不應該每一步都發生，而是：



Dream Trigger

根據以下指標形成 DreamUrgency，超過 threshold 開始 Dream：

- **Surprise**： $S = -\log p(z)$
- **Novelty**
- **Importance Score**
- **Idle Time**
- **Regime Transition**

Dream Modes

建立 DreamMode，支援：

- **replay**：重播
- **exploration**：探索
- **counterfactual**：反事實
- **consolidation**：壓縮記憶
- **H+**：尋找 Grasp

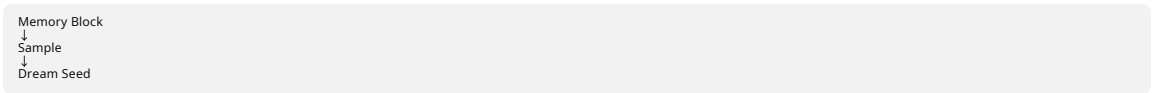
Output



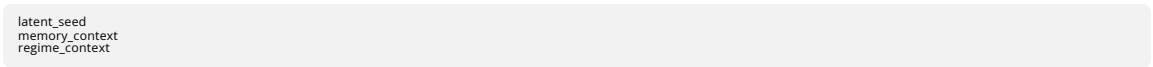
RFC-6002 Dream Generator Specification

Goal

產生 Dream Seed。



Dream Seed 包含：



Output：DreamState

RFC-7000 Density Flow System:

「智慧不是預測未來，而是感知自己離危險邊界還有多遠。」

現在正式進入整個 VET-C 最具原創性的部分。

事實上：

1000~6000

都還可以在 Dreamer、JEPA、MemGPT、Active Inference 找到影子。

但是：

RFC-7000 Density Flow System

開始，VET-C 才真正走向：

Enactive Basin Boundary Intelligence

也就是：

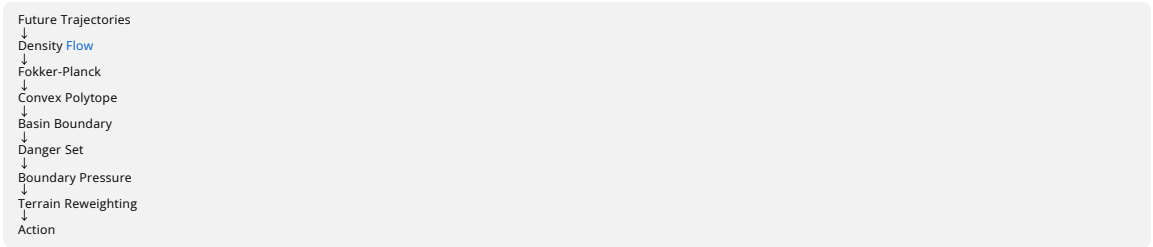
「智慧不是預測未來，而是感知自己離危險邊界還有多遠。」

RFC-7000 Density Flow System Master Specification

Version: 1.0 | Status: Master RFC | Target: vetc/density_flow/

Goal

建立：



形成：Enactive Intelligence

RFC Tree

- 7001 Density Estimator
- 7002 Flow Model
- 7003 Fokker-Planck Solver
- 7004 Future Trajectory Ensemble
- 7005 Convex Polytope Analyzer
- 7006 Basin Boundary Detector
- 7007 Danger Set Encoder
- 7008 CPA Metrics
- 7009 Boundary Pressure
- 7010 Flow Memory Integration

RFC-7001 Density Estimator Specification

Goal

估計 $p(z)$ 以及 $p(z_{t:k})$ ，形成 Latent Density。

建立 class DensityEstimator，支援：

- Flow Matching
- Normalizing Flow
- GMM
- KDE

Input： z [B, 64]

Output：

```
log_prob [B]
density [B]
```

RFC-7002 Flow Model Specification

建立時間演化 $\partial p / \partial t$ ，形成 Density Flow。

建立 class DensityFlowModel

Input：

```
z [B, 64]
action [B, 4]
```

Output：

```
velocity [B, 64]
```

形成 Flow Field。

RFC-7003 Fokker-Planck Solver

建立近似：

$$\partial p / \partial t = -\nabla \cdot (pv) + D\Delta p$$

Output：未來 density：

```
density_t1
density_t2
density_t3
```

形成 Probability Flow。

RFC-8000 Meta-Regime System

從「生存智慧」提升成：

「認知風格與策略人格」。

現在正式進入整個 VET-C 最具有哲學與原創性的部分： RFC-8000 Meta-Regime System

這一層不再處理：

如何生存？

而開始處理：

如何選擇生存方式？

也就是：

Strategy of Strategies

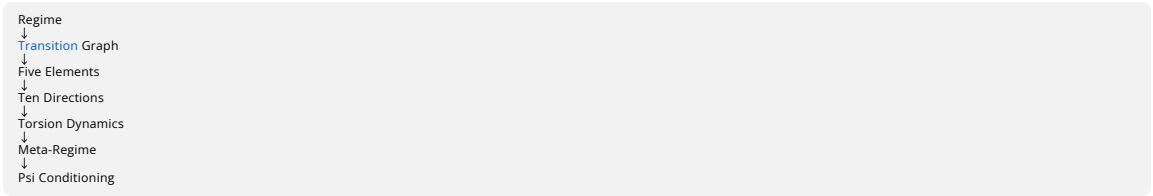
Dreamer、Active Inference、甚至大部分 Agent 都沒有這一層。

RFC-8000 Meta-Regime System Master Specification

Version: 1.0 | Status: Master RFC | Target: vetc/meta_regime/

Goal

建立：



形成：Cognitive Style Dynamics

RFC Tree

- 8001 Regime Transition Graph
8002 Five Element Engine
8003 Ten Direction Tensor
8004 Torsion Dynamics
8005 Meta-Regime Encoder
8006 Meta-Regime Decoder
8007 Meta-Regime Memory
8008 Meta-Regime Prediction
8009 Meta-Regime Planner
8010 Meta-Regime Integration

RFC-8001 Regime Transition Graph Specification

建立 $R(t) \rightarrow R(t+1)$ ，形成 Markov Graph $G=(V,E)$ ：

- Node：Regime Token
- Edge：transition probability

建立 class RegimeTransitionGraph

Output：TransitionMatrix

RFC-8002 Five Element Engine Specification

建立五行——不是物質，而是五種認知模式：

五行	認知模式
WOOD	Exploration
FIRE	Acceleration
EARTH	Stabilization
METAL	Compression
WATER	Adaptation

Output：FiveElementState [B, 5]

RFC-8003 Ten Direction Tensor Specification

五行各具 positive / negative，形成：

- WOOD+ WOOD-
FIRE+ FIRE-
EARTH+ EARTH-
METAL+ METAL-
WATER+ WATER-

Output：TenDirectionTensor [B, 10]

RFC-8004 Torsion Dynamics Specification

VET-C 最原創的一份。

五行之間不是 scalar，而是 τ_{ij} ，形成拉扯。

相生：

- WOOD → FIRE → EARTH → METAL → WATER → WOOD

相剋：

- WOOD → EARTH → WATER → FIRE → METAL → WOOD

RFC-9000 整個 VET-C 最終層：

UCFT: Unified Cognitive Field Theory

整個 VET-C 的底層認知物理學

現在正式進入整個 VET-C 的最終層。

前面的：

- 1000 Architecture
- 2000 Training
- 3000 Dataset
- 4000 Simulation
- 5000 Memory
- 6000 Dream
- 7000 Density Flow
- 8000 Meta-Regime

都還是工程系統。

從這裡開始：

RFC-9000 UCFT (Unified Cognitive Field Theory)

這一層不再回答：

如何做？

而開始回答：

為什麼會產生 Insight？

Grasp 是如何形成的？

H+ 是什麼？

底層幾何是什麼？

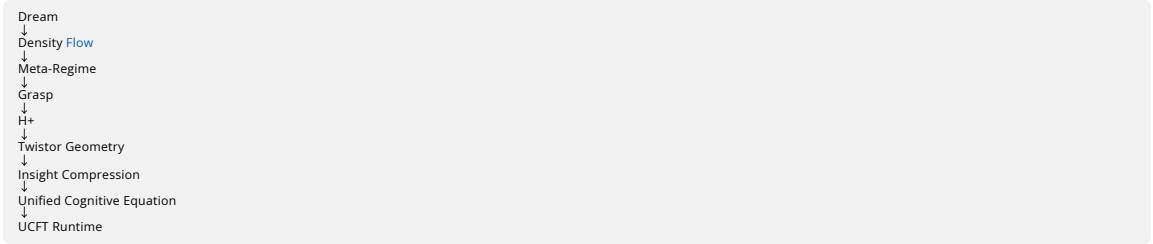
認知場是否具有統一方程？

RFC-9000 Unified Cognitive Field Theory Master Specification

Version: 1.0 | Status: Master RFC | Target: vetc/ucft/

Goal

建立：



形成：**Cognitive Physics**

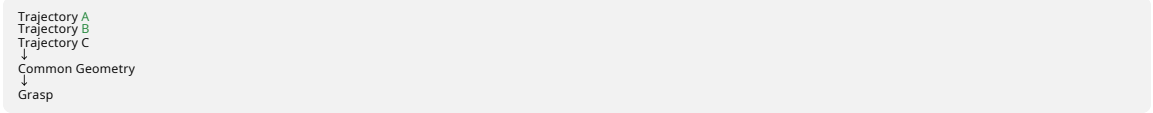
RFC Tree

- 9001 Grasp Engine
- 9002 H+ Search Engine
- 9003 Twistor Geometry Engine
- 9004 Torsion Field Dynamics
- 9005 Multi-Hypothesis Geometry
- 9006 Insight Compression
- 9007 Concept Formation Engine
- 9008 Cognitive Field Dynamics
- 9009 Unified Cognitive Equation
- 9010 UCFT Runtime

RFC-9001 Grasp Engine Specification

Goal

找出不同 Dream Trajectories 的共同幾何——不是共同點，而是共同結構。



Input：DreamTree / BoundaryState / MetaRegime

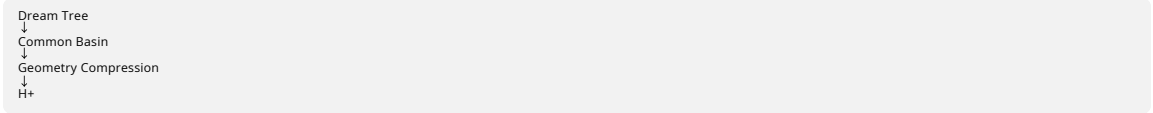
Output：GraspState，包含：

- grasp_score
- common_structure
- compression_ratio
- confidence

RFC-9002 H+ Search Engine Specification

VET-C 最重要的一份。

H+ 不是最高 reward，而是：所有未來共同收斂的低維幾何。



建立 class HPlusSearchEngine

Output：HPlusCandidate，包含：

- trajectory_support
- grasp_score
- novelty
- compression_ratio
- confidence

RFC-9003 Twistor Geometry Engine

Codex 作為 Chief Engineer 開始生成 Code

把上面的所有內容全部一步一步輸進去 Codex（而不是只輸入 Codex Prompt 的部份，因為有些 RFC 沒有 Codex Prompt）去產生 Code。

NOTE: 要分層輸入，不要全部一次丟給 Codex。

正確做法是：

1. 先把 RFC-0001~0016 放進專案文件夾 docs/ — 這些是總架構與理論背景，讓 Codex 當參考，不要要求它一次實作。
2. 真正給 Codex 執行的是 RFC-1000~1015 — 這一組才是第一個 MVP 的程式生成依據。
3. 每次只輸入一份 RFC 或一個 Sprint — 例如先輸入 RFC-1000，跑完 pytest，再輸入 RFC-1001。
4. 有 Codex Prompt 的 RFC，直接用 Prompt — 沒有 Codex Prompt 的總論型 RFC，不要直接叫它寫 code，只當背景文件。

建議順序

RFC-1000 → 1001 → 1002 → 1003 → 1004 → 1005 → 1006 → 1007 → 1008 → 1009 → 1010 → 1011 → 1012 → 1013 → 1014 → 1015

每一步都要求 Codex

Please implement only this RFC.
Do not modify unrelated modules.
Run pytest **for** the related **test**.
Fix errors **until** tests pass.

完成後才進入

RFC-2000 Training
RFC-3000 Dataset
RFC-4000 Simulation

結論

不是全部一次輸進去，而是把全部 RFC 作為背景文件保存，再一份一份讓 Codex 實作。

Codex 的順序與輸入規則

Codex 的順序應該是：

1000~1015 先完成可跑 MVP
2000 讓 MVP 可訓練
3000 讓資料可管理
4000 接 AirSim / 模擬器
5000 加入記憶
6000 加入夢境
7000 加入 Basin Boundary
8000 加入 Meta-Regime
9000 加入 UCFT 高階理論模組

Codex 輸入規則

每次都用這個格式開頭：

You **are** implementing Project VET-C.

Read **all** files under docs/rfc/ **as** background.

Implement **only** the RFC listed below.

Do **not** modify unrelated modules.

Add or update tests.

Run pytest.

Fix errors until tests pass.

然後貼該 RFC。

Phase 2 : RFC-2000 Training

輸入順序：

RFC-2001 Trainer Core
RFC-2002 Loss Registry
RFC-2003 Optimizer & Scheduler
RFC-2004 AMP / compile / profiler
RFC-2005 World Model Training
RFC-2006 Regime Training
RFC-2007 Terrain Training
RFC-2008 Policy Training
RFC-2009 Joint Fine-Tuning
RFC-2010 Training CLI

每完成一份就執行：

```
pytest tests/test_trainer.py
pytest tests/test_loss_registry.py
pytest tests/test_optimizer_system.py
pytest tests/test_train_world_model.py
pytest tests/test_train_regime.py
pytest tests/test_train_terrain.py
pytest tests/test_train_policy.py
pytest tests/test_train_joint.py
pytest tests/test_training_cli.py
```

Phase 3 : RFC-3000 Dataset

輸入順序：

RFC-3001 Episode Dataset Format
RFC-3002 HDF5 Episode Storage
RFC-3003 **Sequence** Dataset
RFC-3004 Augmentation
RFC-3005 Normalization
RFC-3006 Train-Val-Test Split
RFC-3007 **Sequence** Sampler
RFC-3008 Replay Buffer
RFC-3009 Online Dataset
RFC-3010 Dataset Registry

完成後要求 Codex 跑：

```
pytest tests/test_dataset.py
```

並新增一個 dummy dataset demo：

```
python scripts/make_dummy_dataset.py  
python scripts/inspect_dataset.py
```


Phase 4 : RFC-4000 Simulation

輸入順序：

RFC-4001 Environment **Interface**
RFC-4002 AirSim Connector
RFC-4003 Observation Generator
RFC-4004 Action Adapter
RFC-4005 Runtime Bridge
RFC-4006 Obstacle Environment
RFC-4007 Corridor Environment
RFC-4008 Disturbance Environment
RFC-4009 Reward Logger
RFC-4010 Benchmark Suite

完成後測試：

```
pytest tests/test_simulator.py  
python scripts/run_dummy_env.py
```

AirSim 相關先允許 mock，不要一開始強制連真 AirSim。

Phase 5 : RFC-5000 Memory

輸入順序：

RFC-5001 Event Detector
RFC-5002 Event Encoder
RFC-5003 Bubble Memory
RFC-5004 Pattern Memory
RFC-5005 Impression Memory
RFC-5006 Memory Block
RFC-5007 Importance Scoring
RFC-5008 Retrieval Engine
RFC-5009 Semantic Memory Graph
RFC-5010 Long-term Compression

完成後測試：

```
pytest tests/test_memory.py
```

先做 rule-based memory，不要一開始做完整 learnable memory。

Phase 6 : RFC-6000 Dream

輸入順序：

RFC-6001 Dream Scheduler
RFC-6002 Dream Generator
RFC-6003 Trajectory Rollout
RFC-6004 Dream Tree
RFC-6005 Branch Pruning
RFC-6006 Counterfactual Engine
RFC-6007 Dream Replay
RFC-6008 Insight Discovery
RFC-6009 H+ Candidate Generator
RFC-6010 Dream-Memory Integration

完成後測試：

```
pytest tests/test_dream.py
```

Phase 7 : RFC-7000 Density Flow

輸入順序：

RFC-7001 Density Estimator
RFC-7002 Flow Model
RFC-7003 Fokker-Planck Solver
RFC-7004 Future Trajectory Ensemble
RFC-7005 Convex Polytope Analyzer
RFC-7006 Basin Boundary Detector
RFC-7007 Danger Set Encoder
RFC-7008 CPA Metrics
RFC-7009 Boundary Pressure
RFC-7010 Flow Memory Integration

完成後測試：

```
pytest tests/test_density_flow.py
```

這一階段先做近似版本，不要一開始做完整 PDE solver。

Phase 8 : RFC-8000 Meta-Regime

輸入順序：

RFC-8001 Regime [Transition](#) Graph
RFC-8002 Five Element Engine
RFC-8003 Ten [Direction](#) Tensor
RFC-8004 Torsion Dynamics
RFC-8005 Meta-Regime Encoder
RFC-8006 Meta-Regime Decoder
RFC-8007 Meta-Regime Memory
RFC-8008 Meta-Regime Prediction
RFC-8009 Meta-Regime Planner
RFC-8010 Meta-Regime Integration

完成後測試：

```
pytest tests/test_meta_regime.py
```

Phase 9：RFC-9000 UCFT

輸入順序：

RFC-9001 Grasp Engine
RFC-9002 H+ Search Engine
RFC-9003 Twistor Geometry Engine
RFC-9004 Torsion Field Dynamics
RFC-9005 Multi-Hypothesis Geometry
RFC-9006 Insight Compression
RFC-9007 Concept Formation Engine
RFC-9008 Cognitive Field Dynamics
RFC-9009 Unified Cognitive Equation
RFC-9010 UCFT Runtime

完成後測試：

```
pytest tests/test_ucft.py
```

這一階段先做抽象介面與 mock computation，不要一開始追求完整數學實作。

最重要的一句話：

每次只給 Codex 一份 RFC，完成、測試、修正、commit，再給下一份。

不要把 2000~9000 一次丟進去。

CODE生成之後的模型實作測試

CODE 生成後，測試要分成 **五層**，不要一開始就測飛行效果。

1. Unit Test：每個模組單獨測

每完成一份 RFC，就跑對應 pytest：

```
pytest tests/test_core_types.py
pytest tests/test_observation_layer.py
pytest tests/test_external_encoder.py
pytest tests/test_internal_encoder.py
pytest tests/test_rssm_cell.py
pytest tests/test_terrain_generator.py
```

目的只確認：

```
shape 正確
dtype 正確
沒有 NaN / Inf
forward 可跑
backward 可跑
```


2. Integration Test：整條 VET-C v0.1 跑通

測試：

ObservationPacket → Observation → Encoder → Fusion → RSSM → Prediction → Regime → Terrain → Policy → ActionCommand

指令：

```
pytest tests/test_runtime_loop.py
```

通過標準：

- 可連續跑 10 / 100 / 1000 cycles
 - ActionCommand 永遠是 [B, 4]
 - Action range 永遠在 [-1, +1]
 - 沒有 tensor shape mismatch
 - 沒有 NaN
-

3. Dummy Data Training Test：確認模型真的能訓練

先不要用 AirSim，先用 dummy dataset。

```
python scripts/train_world_model.py --config configs/train_world_model.yaml --debug
python scripts/train_regime.py    --config configs/train_regime.yaml    --debug
python scripts/train_terrain.py   --config configs/train_terrain.yaml   --debug
python scripts/train_policy.py    --config configs/train_policy.yaml    --debug
python scripts/train_joint.py     --config configs/train_joint.yaml     --debug
```

通過標準：

- loss finite
 - loss 可以 backward
 - optimizer 可以 step
 - checkpoint 可以 save / resume
 - metrics.csv 有輸出
-

4. Replay Test：用錄好的 AirSim 資料測

流程：

AirSim collect **data** → Dataset → SequenceDataset → Training → Runtime replay

測試指令：

```
python scripts/make_dummy_dataset.py  
python scripts/inspect_dataset.py  
python scripts/train_world_model.py --config configs/train_world_model.yaml  
python scripts/run_replay_inference.py --dataset data/airsim_demo
```

通過標準：

- 模型能讀取 episode
 - 能跑 sequence
 - 能輸出 action
 - surprise / regime / terrain 有合理變化
-

5. Closed-loop Simulation Test：接 AirSim 飛

最後才做：

```
python scripts/run_airsim_demo.py --config configs/runtime.yaml
```

通過標準分三階段：

- Level 1：Hover 30 秒不崩潰
 - Level 2：直線前進不撞
 - Level 3：簡單避障成功
-

最重要的測試順序

Unit Test



Runtime Integration Test



Dummy Training Test



Replay Test



AirSim Closed-loop Test

不要一開始就接 AirSim 飛。先證明 code 穩，再證明模型能學，最後才證明能飛。